PQ Guide

Post-quantum Zcash: the cross-layer assumption inventory, the reversibility divide, and the migration surface

m@rek.onl

Abstract

This volume of *The Zcash Arboretum* assembles the post-quantum picture that no single layer can see. It inventories every deployed cryptographic surface — the shielded core, the transparent layer, the Sprout and Sapling remnants, proof of work, and the frontier protocols — against the two quantum algorithms that matter, and sorts every exposure along one axis: *retroactive* (damage accruing now, in recorded data) against *prospective* (exploitable only once a quantum adversary exists, and mitigable by switching off in time). It quantifies quantum mining through the *Consensus Guide*'s own tables, situates ZIP-2005's recoverability-not-resistance strategy and its stated gaps, computes the migration arithmetic against the published NIST standards, and registers the distance between public statements and the repositories — which contain, at the pinned commits, no deployed post-quantum cryptography at all. The Orchard-core analysis is the *Ironwood Guide*'s; this volume cites it and extends the frame to everything else.

Contents

1 Scope, method, and the two clocks

Every volume below this one proves what an assumption buys while it holds. This volume asks the cross-layer question: what happens to each purchase when the assumption falls to a quantum adversary — and, since the falls are not simultaneous in their *effects*, in what order the damage arrives. The treatment is an inventory and a synthesis, not a re-derivation: the deep analysis of the Orchard core is the *Ironwood Guide*'s (§"Post-quantum security of the Orchard protocol"), the cryptographic definitions are the *Crypto Guide*'s, and this volume constructs only what no lower volume owns — the cross-layer inventory, the quantum-mining analysis, and the migration arithmetic.

Two quantum algorithms carry the whole subject. Shor's algorithm solves the discrete-logarithm problem in polynomial time in *any* group, elliptic curves included, taking DLP, CDH, and DDH with it (*Crypto Guide*, §"The quantum caveat: Shor's algorithm"); every discrete-log-based object in Zcash — which is nearly every asymmetric object — fails completely against it. Grover's algorithm gives a quadratic speedup on unstructured search and nothing more: symmetric primitives and hashes retain half their security exponent, a margin the deployed 256-bit parameters absorb. The standing premise, inherited from the same section, is that no cryptographically relevant quantum computer (CRQC) exists today; nothing in this volume predicts when one will, and every claim is conditioned explicitly on that event.

1.1 The reversibility axis

One distinction organises everything, and the series already owns its two halves. The *Crypto Guide*'s remark "Two regimes, two failure modes" (§"Incompatibility of perfect hiding and perfect binding") observes the asymmetry: a computationally *binding* commitment leaves a future solver able to change a committed value, while a computationally *hiding* one loses everlasting secrecy the day the assumption falls, for everything ever recorded. Which direction each failure *points in time* is the *Ironwood Guide*'s operational recasting, and it turns the asymmetry into a test: a break is *prospective* when its exploitation requires a validator to *accept* a forged object, because acceptance happens at the current chain height — disabling the protocol first removes every future acceptance event, so forking in time is a complete mitigation; a break is *retroactive* when recorded data itself becomes exploitable, and no consensus change can help, because the adversary's copy of the chain is beyond the protocol's reach.

Along that axis, the deployed protocol has exactly one retroactive break — harvest-now-decrypt-later against note encryption — and a stack of prospective ones; Section 2 walks the full inventory. Figure 1 draws the divide.

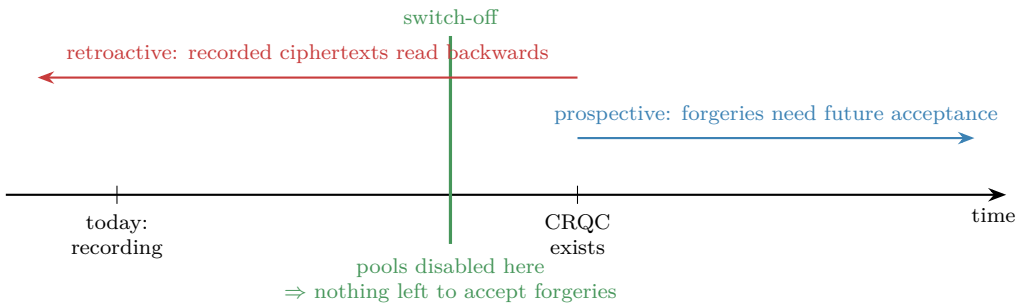

Figure 1: The reversibility divide. A prospective break (blue) needs a validator to accept a forged object after a CRQC exists; switching the vulnerable protocols off first (green) leaves it nothing to act on. The retroactive channel (red) runs the other way: ciphertexts recorded today are read by tomorrow's adversary, and no protocol change reaches the adversary's copy of the chain.

1.2 Sources and their standing

Table 1 lists the sources. Normative weight is ZIP-2005's (Proposed) and the cited Final ZIPs'; the NIST standards are normative for their own schemes and serve here only as the published size and security baselines; everything else is analysis or code. Worked numbers are computed by script (`.wip/pq-drafts/compute.py`, whose output is the source of every numeric claim here); deployed-behaviour claims cite crate, file, and line at the pinned commits.

Source	Role	Pin
ZIP-2005 (Ironwood Quantum Recoverability)	Proposed	`zips 69610984`
ZIPs 213, 229, 258, 326; ZIP-209	normative/draft	same pin
FIPS 203/204/205 (ML-KEM, ML-DSA, SLH-DSA)	published 2024-08-13	retrieved 2026-08-19
Ben-Sasson et al., ePrint 2018/046; ethSTARK 2021/582	analysis	retrieved 2026-08-19
Aggarwal et al., arXiv:1710.10377	analysis (2017)	retrieved 2026-08-19
zebra	deployed validator	`8e9ff3b2c`
reddsa 0.5.1 / halo2_proofs 0.3.2 / orchard 0.15.3	deployed crates	Cargo.lock
zebra-crosslink	prototype	`6d02a1b8`

Table 1: Sources. The quantum-relevant repository state was swept at these pins: the zips repository and the protocol specification contain zero occurrences of ML-KEM, Kyber, ML-DSA, Dilithium, SPHINCS, Falcon, or "lattice"; the sole post-quantum bytes in any ecosystem checkout are an inert, feature-gated transitive dependency (Section 6.3).

2 The inventory: every deployed surface, sorted

Table 2 is the volume's spine: every cryptographic surface a mainnet validator enforces today (with the one frontier prototype, Crosslink, marked as such), its assumption, what a CRQC does to it, and on which side of the divide the damage falls. The Orchard-core rows compress the *Ironwood Guide*'s five-subsection analysis and the committed inventory it rests on; the remaining rows are this volume's. The subsections walk the table by failure mode.

2.1 The retroactive wound

One row of Table 2 is red, and it is the one accruing damage today. Every shielded output ever recorded carries an ephemeral key and a ciphertext; for the Orchard pools, one Shor logarithm per *address* yields the incoming viewing key ($\mathsf{ivk} = \log_{\mathsf{g_d}} \mathsf{pk_d}$), after which the adversary runs the recipient's own trial-decryption loop over the whole recorded chain — amounts, memos, and the complete receiving history of that address, forever. The full anatomy, and the two structural limits — the attack is keyed by the address, and it confers viewing rather than spending — are the *Ironwood Guide*'s (§"The retroactive break: harvest now, decrypt later against note encryption"). Three extensions belong to the cross-layer frame:

- *Every pool, not one.* Sapling and Sprout note encryption are equally DH-keyed and their ciphertexts equally recorded; ZIP-2005 itself states that note encryption for the "Ironwood, Orchard, Sapling, and Sprout pools" is subject to harvest-now-decrypt-later given ciphertext and receiver address. The corpus is the union of every shielded output since 2016, growing with each block — including Ironwood's, whose recoverability machinery deliberately excludes privacy (Section 5).
- *Where the address is already public, so is the history.* Shielded coinbase outputs must decrypt under the all-zero outgoing viewing key (*Ironwood Guide*, §"How value enters the pool"), so their contents were never confidential; and ZIP-213 itself advises miners who rely on the "post-quantum privacy" of an address to use a coinbase-specific one — the ZIP's only pre-2005 use of the phrase.

Surface	Assumption	Consequence of a CRQC	Side
Note encryption (all shielded pools)	DH on the pool's curve	decrypt every recorded ciphertext to a *known address*: amounts, memos, full receiving history	**retro**
Sinsemilla commitments (NoteCommit, $\mathsf{Commit}^{\mathsf{ivk}}$, MerkleCRH)	Pallas DLP (binding)	equivocate openings: forge notes, fake Merkle paths, Spendability attacks	prosp.
Value commitments + binding signature	Pallas DLP	reopen cv to any value; forge balance — silent inflation, bounded only by the ZIP-209 turnstiles	prosp.
RedPallas spend authorisation	Pallas DLP (ROM)	forge spend-auth from public rk; with forged proofs, theft	prosp.
Halo 2 IPA proof soundness	Vesta DLP (ROM)	prove arbitrary false Action statements — the most concentrated point	prosp.
Sapling stack (Groth16, RedJubjub, Jubjub DH)	BLS12-381 / Jubjub DLP	one logarithm forges the whole pool's integrity	prosp.
Sprout stack (Groth16, Ed25519 `JoinSplitSig`)	BLS12-381 / Curve25519 DLP	JoinSplit statements and signatures forgeable	prosp.
Transparent scripts (ECDSA, secp256k1)	secp256k1 DLP	spend any output whose public key is exposed	prosp.
Crosslink finalizer votes (prototype; ed25519)	Curve25519 DLP	forge roster signatures — the finality premise itself (§4)	prosp.
Proof of work (Equihash + SHA-256d filter)	generic search	Grover: quadratic speedup, heavily damped (§3)	neither
BLAKE2b backbone, ZIP-32 tree, Poseidon-as-PRF	symmetric / hash	security exponent halves; parameters absorb it	survives

Table 2: The deployed surfaces under a CRQC. "retro" = retroactive (recorded data becomes exploitable; no protocol change helps); "prosp." = prospective (requires future validator acceptance; switch-off before a CRQC is a complete mitigation, with the residues of Section 5). Orchard-core rows: *Ironwood Guide*, §"Post-quantum security of the Orchard protocol".

- *What stays dark even then.* Without the address, the recorded chain leaks nothing to an unbounded adversary: value commitments are perfectly hiding, the spend-auth randomiser masks ask information-theoretically, and proof transcripts are statistically simulatable (*Ironwood Guide*, §"Prospective breaks: the discrete-logarithm integrity stack", closing paragraph; the underlying theorems are the *Crypto Guide*'s and *Halo 2 Guide*'s). The retroactive channel is address-shaped, exactly.

2.2 The prospective integrity stack

Every other broken row fails forward, and for the Orchard core the *Ironwood Guide* already orders the dominoes — Sinsemilla binding, value commitments, RedPallas, and the IPA's knowledge soundness, the last "the single most concentrated failure point" because every other item's exploitation route runs through it. The cross-layer frame adds four members the core analysis does not cover.

The transparent layer. Script signatures are ECDSA over secp256k1 (specification §'Signature'), verified in deployment by the retired zcashd's own C++ interpreter bundled as a crate (libzcash_script, `depend/zcash/src/pubkey.cpp:23--42`, driven from `zebra-script/src/lib.rs:62--76`). Exposure is per-output and public-key-shaped: an address spent from before has its key on chain — forgeable the day a CRQC exists, no acceptance of any new object required beyond an ordinary transaction; an unspent P2PKH or P2SH

output exposes only a hash until its first spend, leaving a between-broadcast-and-confirmation window thereafter (ZIP-2005's own analysis, which also notes P2PK and bare multisig were never produced by Zcash wallets). Two special cases sharpen the picture. The deferred lockbox is secured by consensus accounting alone — the pool has no output, no address, and no key, so a CRQC finds nothing to steal there; only a consensus change moves it. The 78,750 ZEC already disbursed under ZIP-271, by contrast, sit in a 2-of-3 P2SH multisig whose extended public keys are published in the ZIP itself, so hash-hiding provides those particular funds no pre-spend quantum cover.

The Sprout remnant. Version-4 transactions remain consensus-valid — ZIP-2003 (disallowing them) is Draft and absent from the NU6.3 upgrade list — so `zebra` still runs Groth16 JoinSplit verification and Ed25519 `JoinSplitSig` checks (`zebra-consensus/src/transaction.rs:1230, 1265--1268`; `ed25519-zebra`). ZIP-2005 states the consequence plainly: the broken proof system suffices to forge JoinSplit statements, and both Sprout signature layers are forgeable. One logarithm on BLS12-381 undoes Sprout and Sapling integrity alike; one on Pallas/Vesta undoes Orchard and Ironwood.

The Sapling pool. Groth16 over BLS12-381 with RedJubjub authorisation and Jubjub key agreement — the same stack shape as Orchard with every assumption transplanted, and deliberately *excluded* from ZIP-2005's recoverability (its stated analysis-economy decision). Sapling funds therefore face the harder edge of the strategy: prospective integrity failure like Orchard's, but no recovery door — migration before switch-off is the only remedy the design offers.

The finality prototype. Crosslink's roster votes are plain ed25519 signatures under long-lived finalizer keys (Section 4); a CRQC forges the very signatures whose aggregation is supposed to make history *assuredly* final.

2.3 The surviving backbone

What remains standing is exactly what ZIP-2005 builds on. The BLAKE2b expansion registry, the hardened-only ZIP-32 key tree (whose $\geq$256-bit seed requirement the ZIP text motivates by quantum analysis), Poseidon in its PRF role, and the consensus hashes — SHA-256d in the difficulty filter, BLAKE2b in the ZIP-221 history tree — face only generic speedups: Grover halves the security exponent, $2^{256} \to 2^{128}$, a margin the parameters absorb, and the sharper quantum collision algorithms remain memory-bound curiosities (the *Ironwood Guide*'s §"Primitives facing only generic speedups" carries the careful statement, including its best-known-not-provably-best flag). The load-bearing consequence, stated there and relied on everywhere in Section 5: because key derivation is purely symmetric, *ownership remains provable after the assumption that made it enforceable has fallen* — the seed still derives the keys; what is lost is only the DL-based machinery for exercising them.

3 Quantum mining

The one broken row that is neither retroactive nor prospective is proof of work, because Grover attacks it and Grover only halves an exponent. No volume of the series analyses this; the *Consensus Guide* builds the double-spend model on a fixed hashrate share q but never asks what a quantum miner does to q. This section closes that gap, reusing that volume's machinery.

The gated quantity is a hash lottery: a block is valid when the SHA-256d of its header falls below the target (*Consensus Guide*, §"Equihash and the memoryless model"; the difficulty filter is deployed at `zebra-consensus/src/block/check.rs:112--139` over the SHA-256d header hash). That filter is unstructured search, exactly Grover's domain — with three deflations that the honest statement must carry, and a fourth specific to Zcash: the candidates a classical Equihash miner

feeds the filter come from Wagner-structured solver runs, not brute force, so a quantum miner would have to embed that structured search in its Grover oracle to gain on the whole pipeline rather than on the filter alone.

Remark 3.1 (The three deflations)**.** Aggarwal, Brennen, Lee, Santha, and Tomamichel (arXiv:1710.10377, 2017) state them (published in *Ledger*, 2018). *Quadratic, not exponential*: Grover "can perform the hashcash [proof of work] by performing quadratically fewer hashes", so N classical trials become $\sqrt{N}$ — a square root, not a collapse. *Gate speed*: at 2017 difficulty "the extreme speed of current specialized ASIC hardware ... coupled with much slower projected gate speeds for current quantum architectures, essentially negates this quadratic speedup ... giving quantum computers no advantage"; only a hypothesised 100 GHz gate speed would let a quantum miner "solve the [proof of work] about 100 times faster". *Parallelisation*: Grover parallelises badly — across d processors the search *time* falls only as $\sqrt{d}$, against the linear scaling a classical pool enjoys, so a quantum miner cannot buy back the speed by fanning out. The date matters: these are 2017 projections, cited as the structural shape of the advantage, not a forecast.

The model needs no rebuilding, only a reading. A quantum miner is one whose search is faster per unit hardware; fold the advantage into an effective speed multiplier g on the attacker's raw hashing, and the *Consensus Guide*'s hashrate share becomes

$$q_{\text{eff}}(f,g) \;=\; \frac{gf}{gf+(1-f)} \qquad \text{for a raw share } f \text{ of the total hashing rate}$$

(the honest remainder being $1-f$; at $g=1$, $q_{\text{eff}}=f$, the plain hashrate share of that volume), after which every result of that volume applies to q_{eff} unchanged — its double-spend probability $r(q,n)$, its confirmation tables, its majority threshold at $q=\frac{1}{2}$. Table 3 computes the effect (by script, from that volume's $r(q,n)$).

raw f	speedup g	q_{eff}	$r(q_{\text{eff}},6)$	$r(q_{\text{eff}},10)$
1%	1	0.010	$<10^{-6}$	$<10^{-6}$
1%	10	0.092	0.037%	0.0004%
1%	100	0.503	100%	100%
5%	10	0.345	28.0%	16.0%
10%	2	0.182	1.44%	0.15%
10%	10	0.526	100%	100%
25%	2	0.400	49.3%	37.2%

Table 3: A per-unit mining speedup g applied to an attacker's raw share f of total hashing, and the resulting double-spend success against 6 and 10 confirmations, computed by script from the *Consensus Guide*'s $r(q,n)$. The threshold that matters is the majority line: $q_{\text{eff}} > \frac{1}{2}$ iff $g > (1-f)/f$, i.e. a 10% miner needs $g>9$, a 5% miner $g>19$, a 1% miner $g>99$.

The reading is reassuring and precise. Grover halves the search *exponent* — a quadratic search over an H-bit target is $2^{H/2}$ rather than 2^H trials — so the ideal *rate* multiplier grows as the square root of the expected trial count, not to some fixed bound; what keeps the realised g small is not the algorithm but the gate-speed deflation of Remark 3.1 (no advantage at 2017 parameters, about 100 under its hypothesised 100 GHz gates). The table spans that uncertainty deliberately. Read at a modest near-term operating point of $g=2$, even a 25% raw share reaches only $q_{\text{eff}}=0.40$, short of majority, and a 10% share sits at $r=1.44\%$ over six confirmations; read at the 100 GHz hypothesis, a $g=100$ miner is over the majority line from a 1% raw share. Either way a quantum miner rewrites the double-spend arithmetic the way any faster miner does — by raising q — and proof of work degrades gracefully where the signature layers shatter. This asymmetry is the same one Aggarwal et al. draw for Bitcoin: the proof of work is "relatively resistant", the signatures "much more at risk".

Remark 3.2 (Lumpy progress, quantum edition)**.** The *Consensus Guide*'s difficulty remark (§"Difficulty in motion") shows that the overtake probability of a work-scored race depends on the *granularity* of progress, but its mechanism is work *per block* under difficulty manipulation — which quantum mining leaves unchanged, since a quantum miner's blocks carry the same work as anyone else's at the network difficulty. A distinct channel does remain: the *inter-arrival* dispersion of a quantum miner's block discoveries need not match the memoryless stream the model assumes, and its direction (Grover completions may be more regular, not less) is unanalysed here. It is flagged as the one place the reduction to q_{eff} is not exact; the effect is second-order against the deflations above.

4 The frontier surfaces

Two designs beyond the deployed core carry quantum-relevant structure their own volumes state but do not analyse; the closing volume must.

Crosslink finality. The trailing-finality prototype (*Crosslink Guide*, §"Finalizer keys") gives each finalizer a single ed25519 key and makes a roster vote a plain ed25519 signature over a 76-byte payload (`zebra-crosslink/src/lib.rs:2011` onward, at the prototype pin). The construction's whole point is to convert proof-of-work's *probabilistic* finality — the $r(q, n)$ of Section 3 — into *assured* finality by BFT signatures. Under a CRQC that inverts: the finalizer keys are long-lived roster identities that must stay secret indefinitely, and Shor forges their signatures, so an adversary manufactures the two-thirds agreement the BFT layer contributes. The prototype sources no threshold or post-quantum design for these keys anywhere; the *FROST Guide*'s own inference (§"Crosslink finalizer keys") is that the natural fit would be FROST(Ed25519, SHA-512) — itself classical. Crosslink's safety is a disjunction — assured finality from the proof-of-work prefix *or* the BFT agreement (*Crosslink Guide*, §"The safety theorems and their assumptions") — so a CRQC does not erase finality outright; it deletes the BFT half of that disjunction, collapsing the guarantee back onto the probabilistic proof-of-work floor of Section 3. The upgrade the BFT layer was meant to buy is exactly what a quantum adversary forges.

FROST threshold spending. FROST rerandomised RedPallas is the deployed threshold path, and it is DL-based like everything it signs for (reddsa, `src/frost/redpallas.rs`: group Pallas, an aggregated signature verifying as an ordinary RedPallas spend-auth). ZIP-2005 extends its quantum-recovery mechanism to FROST through a shared quantum spending key qsk that every participant holds (*FROST Guide*, §"Custody of the ZIP-2005 quantum spending key") — which defeats the t-of-n threshold against a quantum adversary, since one compromised participant's qsk suffices. The ZIP accordingly recommends migrating FROST-held funds to a genuinely post-quantum threshold protocol "as soon as" one exists; none does, and the derivation-agreement gap (participants must privately agree a seed sk, with no protocol for it) remains open. The frontier inherits the core's strategy and its unfinished edges both.

5 ZIP-2005 in the cross-layer frame

The deployed answer to all of this is one Proposed ZIP, and the *Ironwood Guide* anatomises its mechanism (§"ZIP-2005: quantum recoverability of Ironwood notes") — the `0x0B`-tagged hashed derivation that makes each Ironwood note's commitment *conditionally* post-quantum-binding, the qsk path for non-seed keys, the non-normative Recovery Statement, and the QROM security bounds. This volume adds only the cross-layer placement.

What it is. A recoverability mechanism, not a resistance one: ZIP-2005 states in terms that it "does not by itself make the protocol secure against quantum adversaries", and arranges instead that Ironwood-pool funds survive the eventual, necessary switch-off of the DL-based

pools. Its lever is the surviving backbone of Section 2.3: because the seed still derives every key symmetrically *and* the `0x0B`-tagged hashed derivation makes each Ironwood note commitment conditionally post-quantum-binding, a future Recovery Protocol can let a holder prove ownership and re-mint under post-quantum assumptions — ownership outliving enforceability.

What it is not, mapped onto the inventory (Table 2):

- *The retroactive wound is untouched.* Privacy is an explicit non-requirement; harvest-now-decrypt-later persists for every pool including Ironwood, with mitigations only "under consideration". The one break that is bleeding today gets nothing.
- *Several surfaces are out of scope entirely.* Transparent ECDSA is deferred to a Reserved ZIP-2007 stub; and the Sapling, Sprout, and *legacy-Orchard* (lead-byte-`0x02`) pools get no recoverability at all — their notes are *unrecoverable*, so once those protocols switch off, funds not migrated out are permanently unspendable. For those pools migration is not advice but the only exit.
- *Recovery itself is unbuilt.* The Recovery Protocol is designed-but-unspecified ("subject to change"), and it fixes no post-quantum proof system or commitment-tree hash — a stated requirement to leave them open. The recoverability claim is thus conditional on an artifact that does not yet exist.
- *Timing is load-bearing.* Every prospective mitigation assumes switch-off happens *before* a CRQC; attacks in the exposure window — Spendability roadblocks, spend-auth theft, and the turnstile-bounded inflation that "would not necessarily be detected" — are not repaired retroactively.

The honest summary is the *Ironwood Guide*'s, promoted here to the whole protocol: the strategy is recoverability, not resistance — keep discrete-log cryptography while it stands, arrange that ownership survives its fall, and race to switch the vulnerable pools off before the fall arrives. The strategy answers integrity comprehensively and privacy not at all.

6 The migration surface

Recoverability defers the problem; resistance eventually requires replacing the broken primitives with post-quantum ones, and the replacements are large. This section prices the swap against the published standards, so the cost is a number rather than an intuition.

6.1 What the replacements weigh

The NIST finals (FIPS 203/204/205, published 2024-08-13) fix the sizes. Table 4 lists the relevant tiers beside their classical counterparts as the series states them — the 64-byte RedPallas signature (ZIP-230 signature encodings; *Ironwood Guide*, the v6 transaction table), the 32-byte ephemeral key (*Sync Guide*), the 192-byte Groth16 proof (*Wallet Guide*). The ratios are the story.

Two of the three swaps are merely expensive. Replacing RedPallas with ML-DSA-44 multiplies each signature by nearly 38; replacing the Diffie–Hellman ephemeral key with an ML-KEM-768 ciphertext multiplies it by 34. A one-Action bundle today occupies $820+64+64+(2720+2272) = 5940$ bytes (the OrchardAction encoding, its spend-auth and binding signatures, and the proof, all as the *Ironwood Guide* and *Halo 2 Guide* state them); substituting ML-DSA for the two signatures and an ML-KEM ciphertext for the ephemeral key, but *leaving the proof unchanged*, already brings it to about 11,700 bytes — a 1.97× inflation from the signature and KEM layers alone (by script). The compact-block cost is sharper still: a per-output ephemeral key of 32 bytes becomes 1088, taking the CompactOrchardAction payload from 148 bytes to about 1200, an 8.1× bandwidth multiplier on exactly the wire format the light-client stack was built to keep small (*Sync Guide*).

Classical (deployed)	bytes	Post-quantum (NIST)	bytes	ratio
RedPallas signature	64	ML-DSA-44 signature	2420	37.8×
		ML-DSA-65 signature	3309	51.7×
		SLH-DSA-128s signature	7856	122.8×
Ephemeral key epk	32	ML-KEM-768 ciphertext	1088	34×
		ML-KEM-768 encaps. key	1184	—
Groth16 proof	192	STARK proof (hash-only soundness): ∼hundreds of kB (order; see below)		

Table 4: Post-quantum replacement sizes against deployed counterparts. ML-DSA figures read from FIPS 204 Table 2, SLH-DSA from FIPS 205 Table 2, and ML-KEM from FIPS 203 Table 3; ML-DSA private keys compress to a 32-byte seed. Ratios computed by script.

6.2 The proof problem is the hard one

The third swap is not merely expensive; it is unchosen, and it dominates the size budget. A discrete-log proof system — Halo 2's IPA, or its declared successor Ragu, which is deliberately ECDLP-based (the *Tachyon Guide*'s reading) — has no post-quantum soundness. The established post-quantum route is hash-based: FRI/STARK systems rest, in Ben-Sasson, Bentov, Horesh, and Riabzev's words, on "the existence of collision-resistant hash functions ... for interactive solutions, and ... the random oracle model ... for noninteractive ones" — assumptions "not known to be susceptible to attacks by large-scale quantum computers". The cost is proof size: the same authors note that "ZK-SNARKs are roughly 1000× shorter than ZK-STARK proofs", and the ethSTARK grant milestone compresses a 100,000-hash workload (3.2 MB) to 200 kB at 80-bit security. That 200 kB is three orders of magnitude past a 192-byte Groth16 proof and about forty times a ∼5 kB Halo 2 bundle proof — and a hash-based system is what the Recovery Statement of Section 5 would have to be instantiated in. This is why ZIP-2005 leaves the proof system unchosen: the recoverability design commits to the *shape* of a post-quantum future without paying its size, deferring the largest bill to the day resistance, not recovery, is finally built. The Recovery Statement's in-circuit cost — roughly seven BLAKE2b compressions, a BLAKE3, two Sinsemilla commitments, two Pallas scalar multiplications, and a Merkle path (*Ironwood Guide*) — is small precisely because it re-executes the *surviving* symmetric backbone; the proof *of* that circuit is the part that inflates.

6.3 The register

The series' method is to classify design-stage claims and to cite firsthand; applied to the ecosystem's public posture, the method yields a register worth stating plainly, because the gap it records is large.

In the repositories, at the pinned commits: the zips repository and the protocol specification contain *zero* occurrences of ML-KEM, Kyber, ML-DSA, Dilithium, SPHINCS, SLH-DSA, Falcon, or "lattice". A recursive search for post-quantum scheme names across the ecosystem checkouts finds no protocol code: the only post-quantum crates anywhere are pqcrypto-mlkem and pqcrypto-mldsa in a single `Cargo.lock` (zallet's), pulled transitively and behind a `zcashd-import` feature gate by a wallet-interchange library, and a lone doc comment in the same crate noting ML-KEM work in a dependency's ecosystem — both inert, called by no protocol code. No deployed post-quantum cryptography exists in Zcash at these pins.

In public statements: a claim of a "fully post-quantum" Zcash "in 12–18 months" appears in conference coverage (CoinDesk, 2026-05-08, reporting Swihart at Consensus Miami), and a claim that ML-KEM and ML-DSA are "being tested" in a separate CoinDesk Research long-form piece — neither corresponding to any ZIP, draft, or repository artifact. ZIP-2005 itself is the honest counterweight: it disclaims making the protocol quantum-secure and confines its promise

to recoverability of one pool. This volume records the divergence without adjudicating intent: the design record and the marketing record are simply different documents, and a reader is owed both.

What is genuinely specified, designed, or open, in the series' three bins: the Ironwood note derivations and the seal rules are *specified* (ZIP-2005, NU6.3); the Recovery Protocol and the FROST qsk custody are *designed but unspecified*; and a deployed post-quantum signature for ongoing spends, a chosen post-quantum proof system, a privacy answer to the retroactive wound, and the switch-off schedule itself are *open problems*. The protocol's quantum posture is a well-specified plan to *recover* from a break it has, by deliberate choice, not yet built the means to *prevent*.

7 Relation to the rest of the series

This volume closes the Frontier group and the series. It owns nothing that a lower volume constructs: the quantum caveat, the hiding/binding regimes, and the commitment and proof primitives are the *Crypto Guide*'s and *Halo 2 Guide*'s; the Orchard-core post-quantum analysis and ZIP-2005's mechanism are the *Ironwood Guide*'s; the double-spend arithmetic that Section 3 feeds a quantum miner into is the *Consensus Guide*'s; the qsk custody is the *FROST Guide*'s; the recorded-ciphertext liability and its one structural mitigation are the *Tachyon Guide*'s; the finalizer keys are the *Crosslink Guide*'s. What this volume adds is the join: the single table that places every layer's exposure on one axis, the quantum-mining analysis no volume owned, the migration arithmetic, and the register of what is built against what is said. Read from below, the series proves what each assumption buys; read from here, it shows what is owed when the bill comes due — and that Zcash has chosen to keep the assumptions, insure the funds against their fall, and run for the exit before it comes.